

ФУНДАМЕНТ IT · ДЕНЬ 27 ИЗ 30

Чистый код и code review

Пишем код, который сможет понять другой человек и будущий вы.

Главная идея

Читаемость важнее хитрости. Имена должны объяснять смысл, функции - делать одну понятную вещь, а комментарии - отвечать "почему", если это нельзя выр...

```
def calculate_total(price, quantity):  
    return price * quantity
```

```
# хорошее имя объясняет смысл
```

План урока

Один блок теории - сразу практика и проверка.

1

Понять смысл

2

Выучить
термины

3

Разобрать код

4

Попробовать
самому

5

Собрать мини-
задачу

Как заниматься

Не читай как книгу. На практических слайдах остановись и попробуй решить сам.

Смысл темы простыми словами

Сначала бытовая идея, потом код.

Идея 1

Читаемость важнее хитрости. Имена должны объяснять смысл, функции - делать одну понятную вещь, а комментарии - отвечать "почему", если это нельзя выразить кодом.

Идея 2

DRY напоминает не дублировать знание, но раннее объединение непохожих случаев тоже вредит. Refactoring улучшает структуру без изменения внешнего поведения и должен поддерживаться теста...

Идея 3

Code review ищет ошибки, уточняет решения и распространяет знания по команде. Линтер автоматически находит стилевые и часть логических проблем. Technical debt - цена отложенных упрощен...

Термины дня

Эти слова нужно узнавать в коде и документации.

naming

выбор имён, раскрывающих
назначение данных и действий

DRY

принцип устранения повторяющегося
знания

refactoring

улучшение структуры без изменения
поведения

code review

проверка изменений другим
разработчиком

linter

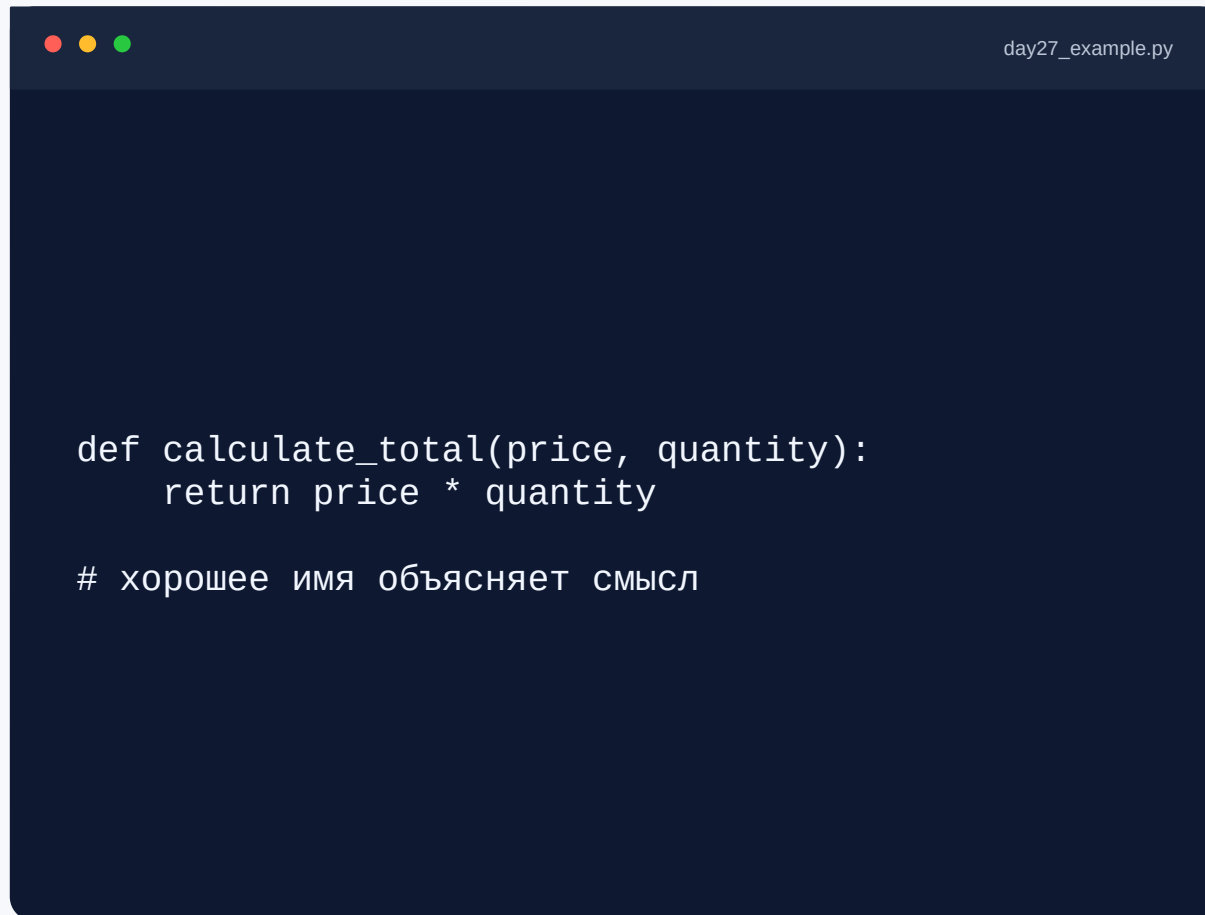
автоматический анализатор стиля и
типовых ошибок

technical debt

будущая стоимость принятых сегодня
упрощений

Код дня

Минимальный пример без лишней сложности.



```
def calculate_total(price, quantity):  
    return price * quantity  
  
# хорошее имя объясняет смысл
```

Что ищем

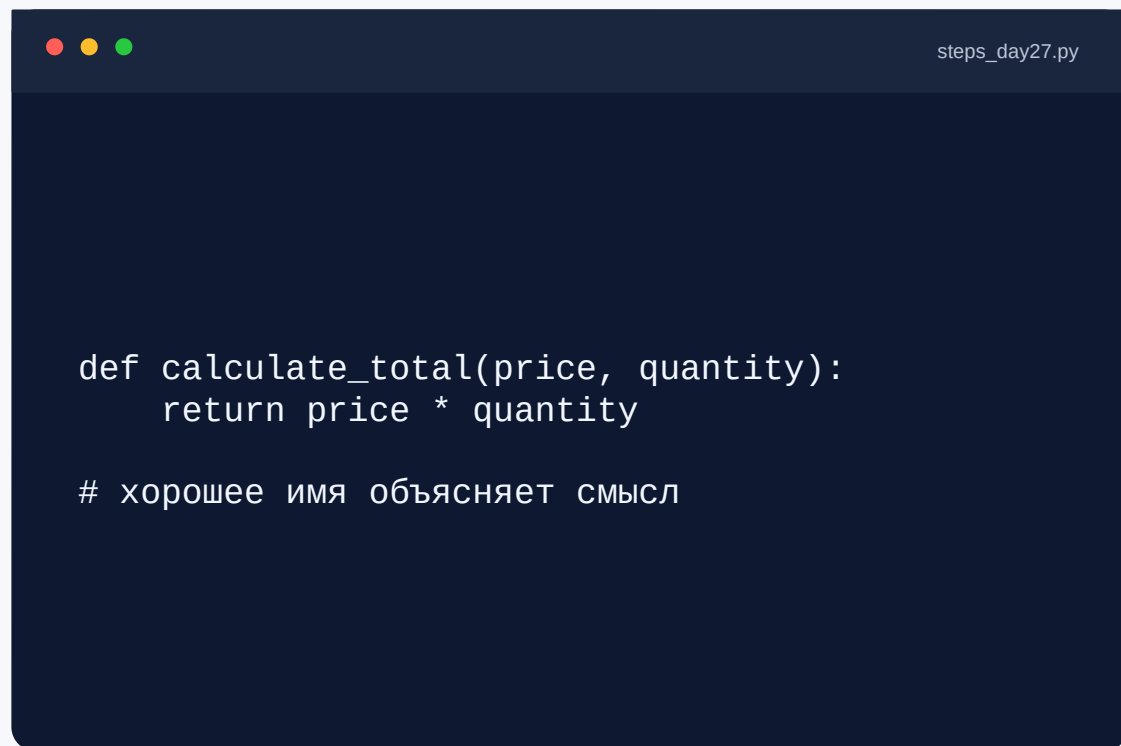
Где данные? Где действие? Где результат?

Главный вопрос

Можешь объяснить каждую строку обычным языком?

Разбор кода по шагам

Не переписываем вслепую.



```
def calculate_total(price, quantity):  
    return price * quantity  
  
# хорошее имя объясняет смысл
```

- Прочитай строку сверху вниз.
- Назови данные, с которыми работает код.
- Объясни, какое действие выполняется.
- Проверь, какой результат должен получиться.

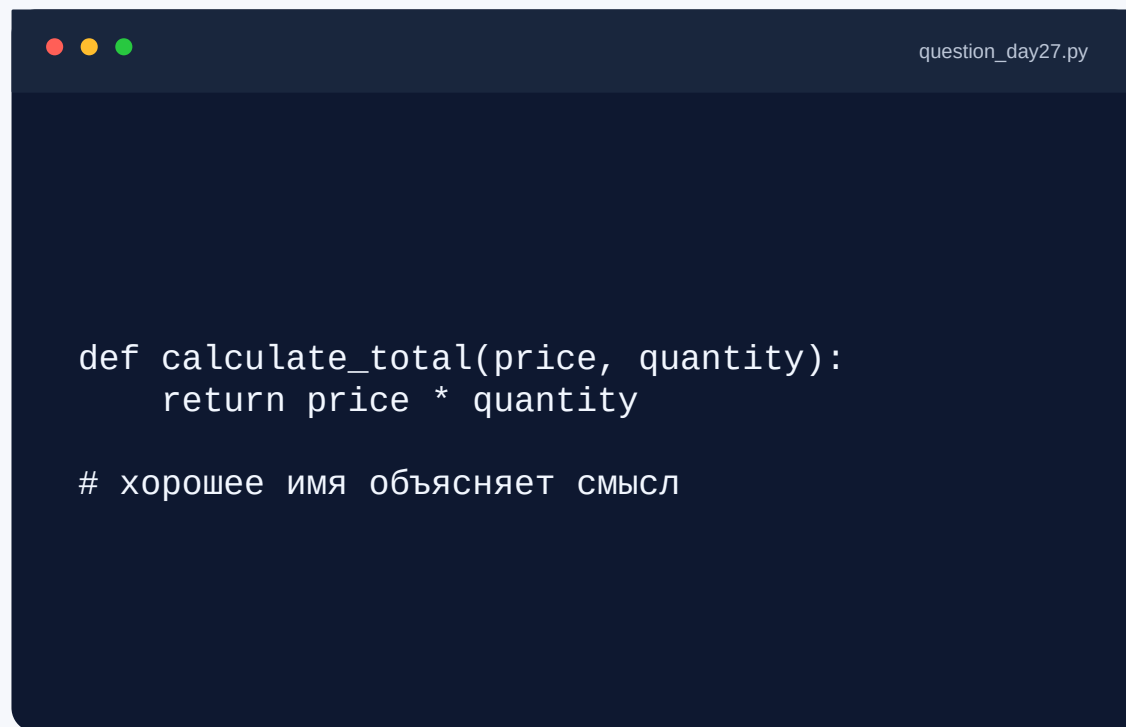


Важно

Если строка непонятна - остановись и переведи ее на обычный язык.

Практика: что произойдет?

Ответь без запуска кода.



```
def calculate_total(price, quantity):  
    return price * quantity  
  
# хорошее имя объясняет смысл
```

Вопрос

Что программа сделает после запуска?

Подсказка

Ищи последовательность: входные данные -> действие -> результат.

Ответ и объяснение

Сравни со своим рассуждением.

Ожидаемая логика

Этот пример относится к теме: Чистый код и code review. Пишем код, который сможет понять другой человек и будущий вы.

Почему так?

Python выполняет инструкции по порядку. Поэтому важно читать код сверху вниз.

Проверка себя

Если можешь объяснить без подсказки - тема закрепляется.

Практика: сделай руками

Небольшое задание по теме дня.

Задание

Переименуй плохие переменные `x`, `a`, `do_it`.

Шаг 1

Создай простой файл или черновик.

Шаг 2

Напиши минимальный рабочий вариант.

Шаг 3

Запусти и проверь результат.

Мини-тест

Выбери ответ и объясни почему.

Вопрос

Что должно сохраняться при рефакторинге?

A

Внешнее поведение программы

B

Количество строк любой ценой

C

Все старые имена переменных

D

Каждая внутренняя функция

Ответ мини-теста

Проверяем не память, а понимание.

Правильный ответ

А: Внешнее поведение программы

Объяснение

Рефакторинг меняет внутреннюю структуру, сохраняя наблюдаемое поведение.

Частая ошибка новичка

Лучше заранее увидеть, где обычно ломается.

Ошибка

Пытаться запомнить формулировку, но не понимать, где это видно в коде.

Как исправить

После каждого термина придумай свой пример и найди его в коде.

Правило

Термин считается выученным, когда ты можешь объяснить его на бытовом примере.

Самостоятельная мини-задача

Напиши маленький код или схему почти с нуля.

Условие

Переименуй плохие переменные `x`, `a`, `do_it`.

- Что программа получает?
- Что она должна сделать?
- Какой результат показать?
- Какие термины дня здесь применяются?

Разбор решения

После попытки сравни с этим подходом.

```
def calculate_total(price, quantity):  
    return price * quantity  
  
# хорошее имя объясняет смысл
```

Не один вариант

Твой код может отличаться. Главное - он решает задачу и читается.

Проверка

Запусти пример и объясни результат вслух.

Задание со звездочкой

Немного сложнее, но без новой темы.

Условие

Расширь практику дня: добавь проверку ошибки, 2-3 тестовых примера и короткое объяснение для README.

Подсказка

Не усложняй. Сделай маленькое улучшение и обязательно проверь его на примере.

Итог дня

Что должно остаться после урока.

- объяснять тему "Чистый код и code review" простыми словами
- узнавать главные термины дня
- читать короткий пример кода
- выполнять практику руками
- находить одну типовую ошибку
- формулировать следующий маленький шаг

Домашнее задание

Возьмите старую функцию или пример с плохими именами `x`, `a`, `do_it`. Переименуйте сущности и разделите код так, чтобы комментарий о действиях стал не нужен.